

Análisis

1. Información que debe conservar

Para cada vehículo, independientemente de su categoría, se debe conservar: Placa, marca, modelo, tarifa diaria, estado de disponibilidad. Esto debido a que la placa funciona como identificador del vehículo, por lo que no pueden existir dos vehículos registrados con la misma placa. Además a nivel general, se debe conservar el dinero acumulado de los alquileres que hayan sido confirmados.

Cada categoría requiere información adicional:

Automóvil:

- Cantidad de pasajeros.
- Indicadores de si la transmisión es automática.

Motocicleta:

- Cilindraje expresado en centímetros cúbicos.

Camioneta de carga:

- Capacidad máxima de carga expresada en toneladas. Esta cantidad puede contener decimales.

2. Requisitos funcionales

1) Registrar vehículos

El sistema debe permitir registrar automóviles, motocicletas y camionetas de carga. Al registrar un vehículo se deben solicitar los datos comunes y los datos particulares correspondientes a su categoría. Además, todo vehículo registrado correctamente debe iniciar con estado disponible.

El registro será válido únicamente cuando:

- La placa no esté vacía.
- No exista otro vehículo con la misma placa.
- La tarifa diaria es mayor que cero.
- La cantidad de pasajeros de un automóvil sea mayor que cero.
- El cilindraje de una motocicleta sea mayor que cero.
- La capacidad de carga de una camioneta sea mayor que cero.

2) Consultar la información

El sistema debe permitir visualizar los vehículos registrados, mostrando su información general, sus características particulares según la categoría y su estado de disponibilidad.

3) Buscar un vehículo por placa

El sistema debe permitir localizar un vehículo utilizando su placa. Debido a que la placa identifica de manera única a cada vehículo, una búsqueda debe producir como máximo un vehículo.

Si la placa no existe, se debe informar claramente al usuario y no modificar ningún dato.

4) Cotizar un alquiler

El sistema debe permitir ingresar la placa de un vehículo, una cantidad de días para calcular cuánto costaría alquilarlo y los días deben ser un número entero positivo. El sistema deberá mostrar:

- Características del vehículo.
- Disponibilidad.
- Costo total del alquiler.

Una cotización es solamente una consulta. Por lo tanto, puede realizarse aunque el vehículo esté ocupado y no debe cambiar su disponibilidad ni incrementar los ingresos acumulados.

5) Confirmar un alquiler:

El sistema debe permitir alquilar un vehículo únicamente cuando se encuentre disponible.

Antes de confirmar la operación se debe mostrar el costo total correspondiente a la cantidad de días solicitados.

Si el cliente acepta:

- Se confirma el alquiler.
- El vehículo cambia de disponible a ocupado.
- El monto total del alquiler se agrega a los ingresos acumulados.

Si el cliente cancela:

- El vehículo conserva su estado anterior.
- Los ingresos acumulados no cambian.
- Un vehículo ocupado no puede volver a alquilarse hasta que sea devuelto.

6) Registrar una devolución

El sistema debe permitir registrar la devolución de un vehículo utilizando su placa.

La devolución solamente será válida si:

- La placa corresponde a un vehículo registrado.
- El vehículo se encuentra actualmente ocupado.

Cuando la devolución se realiza correctamente, el vehículo vuelve a quedar disponible.

La devolución no genera un nuevo cobro y tampoco elimina ni disminuye el ingreso obtenido cuando se confirmó el alquiler. Si se intenta devolver un vehículo que ya está disponible, la operación debe rechazarse y el estado del sistema debe permanecer sin cambios.

7) Generar reporte general

El sistema debe proporcionar un reporte que permita conocer:

- Cantidad total de vehículos registrados.
- Cantidad de vehículos disponibles.
- Cantidad de vehículos alquilados por categoría.

- Dinero acumulado por los alquileres confirmados.

Las cotizaciones, alquileres cancelados, intentos fallidos de alquiler y devoluciones no deben incrementar los ingresos.

8) Mantener un menú de consola

El sistema debe presentar un menú que permanezca activo hasta que el usuario seleccione la opción de salir.

Desde este menú se debe poder acceder al registro de vehículos, consulta de flota, cotizaciones, alquileres, devoluciones y reporte general.

9) Manejar entradas inválidas

El programa debe controlar las entradas proporcionadas por el usuario para evitar que un formato incorrecto provoque la terminación inesperada de la aplicación.

Cuando una operación no pueda realizarse, el programa debe mostrar un mensaje claro y conservar la información existente.

3. Reglas para calcular el alquiler

Cada vehículo parte del mismo cálculo base:

Costo base = tarifa diaria × cantidad de días

Sin embargo, el costo final depende de la categoría concreta del vehículo.

1) Automovil:

Un automóvil de transmisión manual solamente paga la tarifa diaria:

Costo = tarifa diaria × días

Cuando el automóvil tiene transmisión automática, se aplica un recargo de Q50.00 por cada día:

Costo = (tarifa diaria + Q50.00) × días

2) Motocicleta:

Toda motocicleta paga inicialmente:

Costo base = tarifa diaria × días

Si su cilindraje es mayor a 250 cc, se agrega un único recargo de Q75.00 por todo el alquiler:

Costo = (tarifa diaria × días) + Q75.00

El recargo no se multiplica por la cantidad de días.

Si el cilindraje es igual o menor a 250 cc, no se aplica ningún recargo.

3) Camioneta de carga:

La camioneta paga su tarifa diaria más un recargo de Q100.00 por cada tonelada de capacidad máxima y por cada día.

Recargo diario = capacidad máxima × Q100.00

Por lo tanto:

Costo = (tarifa diaria × días) + (capacidad máxima × Q100.00 × días)

El recargo depende de la capacidad máxima de la camioneta y no de la cantidad de carga que el cliente realmente transporte.

4. Clases propuestas y responsabilidades

1) Vehículo

Representa los datos y comportamientos que todos los vehículos de RentaMovil tienen en común. Se propone como una clase abstracta porque el sistema trabaja con categorías concretas de vehículos y porque el cálculo del alquiler depende de la categoría.

Atributos

placa:

- Tipo: String
- Visibilidad: private
- Propósito: identificar de manera única al vehículo.

marca:

- Tipo: String
- Visibilidad: private
- Propósito: almacenar la marca del vehículo.

modelo:

- Tipo: String
- Visibilidad: private
- Propósito: almacenar el modelo del vehículo.

tarifaDiaria:

- Tipo: double
- Visibilidad: private
- Propósito: almacenar el costo base por día del vehículo.

disponible:

- Tipo: boolean
- Visibilidad: private
- Propósito: indicar si el vehículo está disponible para ser alquilado.

Constructor

Vehiculo(String placa, String marca, String modelo, double tarifaDiaria)

Visibilidad: public

Parámetros:

- placa: String
- marca: String
- modelo: String
- tarifaDiaria: double

Propósito: inicializar los datos comunes de un vehículo.

Resultado esperado: crear el vehículo con los datos proporcionados y establecer su disponibilidad inicialmente en true.

Métodos

getPlaca()

- Visibilidad: public
- Parámetros: ninguno.
- Retorno: String
- Propósito: obtener la placa del vehículo.

getMarca()

- Visibilidad: public
- Parámetros: ninguno.
- Retorno: String
- Propósito: obtener la marca del vehículo.

getModelo()

- Visibilidad: public
- Parámetros: ninguno.
- Retorno: String
- Propósito: obtener el modelo del vehículo.

getTarifaDiaria()

- Visibilidad: public
- Parámetros: ninguno.
- Retorno: double
- Propósito: obtener la tarifa diaria del vehículo.

isDisponible()

- Visibilidad: public
- Parámetros: ninguno.
- Retorno: boolean
- Propósito: consultar si el vehículo se encuentra disponible.

alquilar()

- Visibilidad: public
- Parámetros: ninguno.
- Retorno: void
- Propósito: cambiar el estado del vehículo de disponible a ocupado.

devolver()

- Visibilidad: public
- Parámetros: ninguno.
- Retorno: void
- Propósito: cambiar el estado del vehículo de ocupado a disponible.

calcularCosto(int dias)

- Visibilidad: public
- Parámetros: dias de tipo int.
- Retorno: double
- Propósito: calcular el costo total del alquiler de acuerdo con la categoría del vehículo.
- Este método es abstracto para que cada subclase implemente su propia regla de cobro.

obtenerInformacion()

- Visibilidad: public
- Parámetros: ninguno.
- Retorno: String
- Propósito: obtener la información general del vehículo.

2) Automovil

La clase Automovil representa los automóviles de la empresa, almacena sus características particulares y aplica la regla de cobro correspondiente. Hereda de Vehiculo.

Atributos

cantidadPasajeros

- Tipo: int
- Visibilidad: private
- Propósito: almacenar la cantidad máxima de pasajeros del automóvil.

automatico

- Tipo: boolean
- Visibilidad: private
- Propósito: indicar si el automóvil posee transmisión automática.

Constructor

Automovil(String placa, String marca, String modelo, double tarifaDiaria, int cantidadPasajeros, boolean automatico)

Visibilidad: public

Parámetros:

- placa: String
- marca: String
- modelo: String
- tarifaDiaria: double
- cantidadPasajeros: int
- automatico: boolean

Propósito: inicializar los datos comunes heredados de Vehiculo y las características particulares del automóvil.

Resultado esperado: crear un automóvil con los valores proporcionados y disponible para alquiler.

Métodos

getCantidadPasajeros()

- Visibilidad: public
- Parámetros: ninguno.
- Retorno: int
- Propósito: obtener la cantidad máxima de pasajeros.

isAutomatico()

- Visibilidad: public

- Parámetros: ninguno.
- Retorno: boolean
- Propósito: consultar si el automóvil tiene transmisión automática.

calcularCosto(int dias)

- Visibilidad: public
- Parámetros: dias de tipo int.
- Retorno: double
- Propósito: calcular el costo del alquiler aplicando un recargo de Q50.00 por día cuando el automóvil sea automático.

obtenerInformacion()

- Visibilidad: public
- Parámetros: ninguno.
- Retorno: String
- Propósito: obtener la información del automóvil incluyendo la cantidad de pasajeros y el tipo de transmisión.

3) Motocicleta

La clase Motocicleta representa las motocicletas de la empresa, almacena su cilindraje y aplica la regla de cobro correspondiente. Hereda de Vehiculo.

Atributo

cilindraje

- Tipo: int
- Visibilidad: private
- Propósito: almacenar el cilindraje de la motocicleta en centímetros cúbicos.

Constructor

Motocicleta(String placa, String marca, String modelo, double tarifaDiaria, int cilindraje)

Visibilidad: public

Parámetros:

- placa: String
- marca: String
- modelo: String

- tarifaDiaria: double
- cilindraje: int

Propósito: inicializar los datos comunes heredados de Vehiculo y el cilindraje de la motocicleta.

Resultado esperado: crear una motocicleta con los valores proporcionados y disponible para alquiler.

Métodos

getCilindraje()

- Visibilidad: public
- Parámetros: ninguno.
- Retorno: int
- Propósito: obtener el cilindraje de la motocicleta.

calcularCosto(int dias)

- Visibilidad: public
- Parámetros: dias de tipo int.
- Retorno: double
- Propósito: calcular el costo del alquiler y agregar Q75.00 cuando el cilindraje sea mayor a 250 cc.

obtenerInformacion()

- Visibilidad: public
- Parámetros: ninguno.
- Retorno: String
- Propósito: obtener la información de la motocicleta incluyendo su cilindraje.

4) CamionetaCarga

La clase CamionetaCarga representa las camionetas de carga, almacena su capacidad máxima y aplica la regla correspondiente para calcular el costo del alquiler. Hereda de Vehiculo.

Atributo

capacidadToneladas

- Tipo: double
- Visibilidad: private
- Propósito: almacenar la capacidad máxima de carga expresada en toneladas.

Se utiliza double porque la capacidad máxima puede contener valores decimales.

Constructor

CamionetaCarga(String placa, String marca, String modelo, double tarifaDiaria, double capacidadToneladas)

Visibilidad: public

Parámetros:

- placa: String
- marca: String
- modelo: String
- tarifaDiaria: double
- capacidadToneladas: double

Propósito: inicializar los datos comunes heredados de Vehiculo y la capacidad máxima de carga.

Resultado esperado: crear una camioneta de carga con los valores proporcionados y disponible para alquiler.

Métodos

getCapacidadToneladas()

- Visibilidad: public
- Parámetros: ninguno.
- Retorno: double
- Propósito: obtener la capacidad máxima de carga.

calcularCosto(int dias)

- Visibilidad: public
- Parámetros: dias de tipo int.
- Retorno: double
- Propósito: calcular el costo del alquiler agregando Q100.00 por cada tonelada de capacidad máxima por cada día.

obtenerInformacion()

- Visibilidad: public
- Parámetros: ninguno.
- Retorno: String
- Propósito: obtener la información de la camioneta incluyendo su capacidad máxima.

5) RentaMovil

La clase RentaMovil administra la flota de vehículos y coordina las operaciones de registro, búsqueda, cotización, alquiler, devolución y generación del reporte general.

Atributos

vehiculos

- Tipo: ArrayList<Vehiculo>
- Visibilidad: private
- Propósito: almacenar todos los vehículos registrados en la empresa.

ingresosAcumulados

- Tipo: double
- Visibilidad: private
- Propósito: almacenar el dinero acumulado proveniente únicamente de alquileres confirmados.

Constructor

RentaMovil()

- Visibilidad: public
- Parámetros: ninguno.
- Propósito: inicializar la administración de la empresa.
- Resultado esperado: crear una colección de vehículos vacía e inicializar los ingresos acumulados en Q0.00.

Métodos

registrarVehiculo(Vehiculo vehiculo)

- Visibilidad: public
- Parámetros: vehiculo de tipo Vehiculo.
- Retorno: boolean
- Propósito: registrar un nuevo vehículo siempre que su placa no se encuentre repetida.
- Resultado esperado: devolver true cuando el registro sea exitoso y false cuando no pueda realizarse.

buscarVehiculo(String placa)

- Visibilidad: public
- Parámetros: placa de tipo String.
- Retorno: Vehiculo
- Propósito: localizar un vehículo mediante su placa.

- Resultado esperado: devolver el objeto Vehiculo correspondiente a la placa o null si no existe un vehículo registrado con esa placa.

cotizar(String placa, int dias)

Visibilidad: public

Parámetros:

- placa: String
- dias: int

Retorno: double

Propósito: obtener el costo del alquiler sin modificar la disponibilidad del vehículo ni los ingresos acumulados.

Resultado esperado: devolver el costo total del alquiler sin modificar la disponibilidad del vehículo ni los ingresos acumulados.

confirmarAlquiler(String placa, int dias)

Visibilidad: public

Parámetros:

- placa: String
- dias: int

Retorno: boolean

Propósito: realizar un alquiler cuando el vehículo exista y esté disponible, cambiar su estado a ocupado y agregar el costo a los ingresos acumulados.

Resultado esperado: devolver true cuando el alquiler se realice correctamente y false cuando sea rechazado.

registrarDevolucion(String placa)

- Visibilidad: public
- Parámetros: placa de tipo String.
- Retorno: boolean
- Propósito: registrar la devolución de un vehículo que se encuentre ocupado.
- Resultado esperado: devolver true cuando la devolución sea válida y false cuando no pueda realizarse.

mostrarFlota()

- Visibilidad: public
- Parámetros: ninguno.
- Retorno: void

- Propósito: mostrar la información de los vehículos registrados.

mostrarReporte()

- Visibilidad: public
- Parámetros: ninguno.
- Retorno: void
- Propósito: mostrar la cantidad total de vehículos, los disponibles, los alquilados por categoría y los ingresos acumulados.

getIngresosAcumulados()

- Visibilidad: public
- Parámetros: ninguno.
- Retorno: double
- Propósito: obtener el total de ingresos acumulados por los alquileres confirmados.

5. Organización de características comunes y particulares

La herencia permite representar de manera directa que un automóvil, una motocicleta y una camioneta de carga son vehículos.

La clase Vehiculo tiene los elementos compartidos: placa, marca, modelo, tarifa diaria, disponibilidad, operaciones relacionadas con el estado del vehículo.

De esta manera, estos elementos no necesitan repetirse en las tres categorías.

Las clases hijas contienen solamente aquello que las diferencia, Automovil agrega pasajeros y transmisión, Motocicleta agrega cilindraje, CamionetaCarga agrega capacidad máxima de carga.

La misma separación se aplica al comportamiento. Todos los vehículos deben poder calcular el costo de un alquiler, pero la manera de calcularlo cambia. Por ello, Vehiculo define que debe existir la operación calcularCosto(int dias), mientras que cada subclase implementa su propia versión. RentaMovil mantiene una relación de composición con Vehiculo, ya que administra la colección de vehículos que forman parte de la flota. Una instancia de RentaMovil puede contener múltiples objetos de tipo Vehiculo, incluyendo objetos Automovil, Motocicleta y CamionetaCarga.

6. Colaboración de los objetos durante un alquiler

Cuando se desea realizar un alquiler, los objetos colaboran de la siguiente manera:

1. Main recibe del usuario la placa y la cantidad de días.
2. RentaMovil localiza el vehículo mediante su placa.
3. Se comprueba que la placa exista.
4. Se comprueba que la cantidad de días sea válida.

5. RentaMovil verifica que el vehículo esté disponible.
6. Se solicita al objeto Vehiculo su costo mediante `calcularCosto(dias)`.
7. Gracias al polimorfismo, la categoría concreta realiza el cálculo correspondiente.
8. El total se muestra antes de confirmar.
9. Si el cliente cancela, no se realiza ningún cambio.
10. Si confirma, el vehículo pasa a ocupado y RentaMovil agrega el costo a `ingresosAcumulados`.

Esta distribución evita que Main tenga que conocer las fórmulas de cobro. Asimismo, RentaMovil coordina la operación, pero delega el cálculo a quien posee la información necesaria: el vehículo concreto.

7. Protección y consistencia del estado

Los atributos de las clases serán privados para evitar modificaciones arbitrarias desde otras partes del programa.

Especialmente, la disponibilidad no debe modificarse directamente desde `Main`. El cambio debe producirse mediante operaciones controladas de alquiler y devolución.

Los ingresos acumulados tampoco deben modificarse durante una cotización. Solamente una confirmación exitosa del alquiler debe incrementar dicho valor.

Por lo tanto:

Cotización: no cambia disponibilidad ni ingresos.

Alquiler cancelado: no cambia disponibilidad ni ingresos.

Alquiler rechazado: no cambia disponibilidad ni ingresos.

Alquiler confirmado: cambia disponible a ocupado e incrementa ingresos.

Devolución válida: cambia ocupado a disponible y conserva ingresos.

Devolución inválida: no cambia estado ni ingresos.

Esto permite mantener consistentes tanto el estado de la flota como el dinero acumulado.

8. Incorporación futura de una nueva categoría

El diseño considera que RentaMovil puede incorporar nuevas categorías de vehículos en el futuro.

Por ejemplo, si posteriormente se agregara una clase Microbus, esta podría extender Vehiculo: `Microbus extends Vehiculo`.

La nueva clase definiría únicamente:

- sus atributos particulares, su constructor, su implementación de `calcularCosto(int dias)`, la información particular que deba mostrar.

Los atributos comunes como placa, marca, modelo, tarifa diaria y disponibilidad continuarían perteneciendo a `Vehiculo`. Además, las operaciones generales de búsqueda y almacenamiento podrían seguir trabajando con objetos `Vehiculo`, por lo que no sería necesario duplicar toda la lógica existente para administrar la nueva categoría.

Sí tendrían que modificarse aquellas partes de la aplicación que dependen explícitamente de las categorías disponibles, principalmente las opciones de registro desde el menú y el reporte si este debe presentar un conteo separado para la nueva categoría.

Esta organización permite extender el sistema manteniendo centralizados los elementos comunes y aislando en cada subclase las características y reglas de cobro que pueden variar.